Layer Variational Quantum Eigensolver (L-VQE)

Taha Çakmak João Martins Jules Dupont

WI4650 Applied Quantum Algorithms (Q3-Q4 2025/26)

Delft University of Technology

June 2026

Outline

1. The L-VQE algorithm
2. Results and reproduction of the original paper
3. Extension 1: the MaxCut problem
4. Extension 2: real hardware (IBM Quantum)
5. Bonus: quantum walk

Variational Quantum Eigensolver (VQE)

A hybrid quantum–classical algorithm. Encode the objective $C(x)$ in a Hamiltonian H with $H|x\rangle = C(x)|x\rangle$; the optimum is the ground state of H.

The optimization loop:

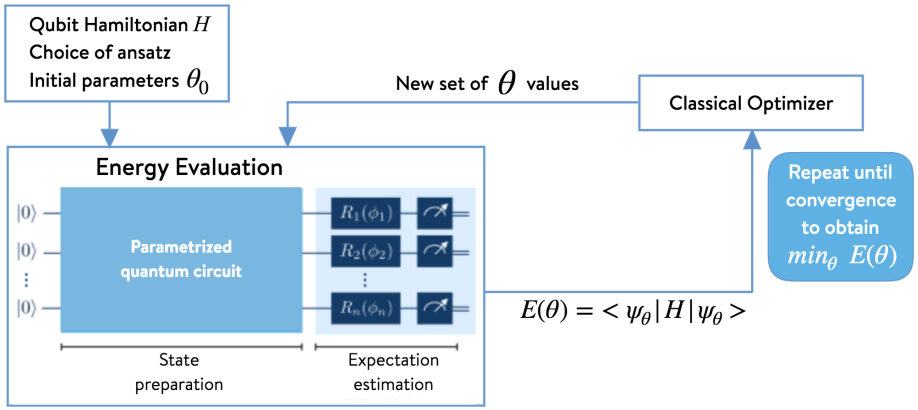
1. *Quantum:* prepare the ansatz

$$|\psi(\theta)\rangle = U(\theta)|0\rangle^{\otimes n}$$

2. *Quantum:* estimate the energy

$$E(\theta) = \langle\psi(\theta)|H|\psi(\theta)\rangle$$

3. *Classical:* update $\theta \to \theta'$ to minimize $E(\theta)$



The Central Question: which ansatz?

VQE is only as good as its ansatz $U(\theta)$.

Expressivity	Trainability
Can it *reach* the solution?	Can the optimizer *find* it?

Hardware-efficient ansätze (single-qubit rotations + CNOTs):

- Shallow and expressive which is good for NISQ
- Follow native hardware connectivity
- But: degrade with size ⇒ **barren plateaus**

Barren Plateaus: trainability does not scale

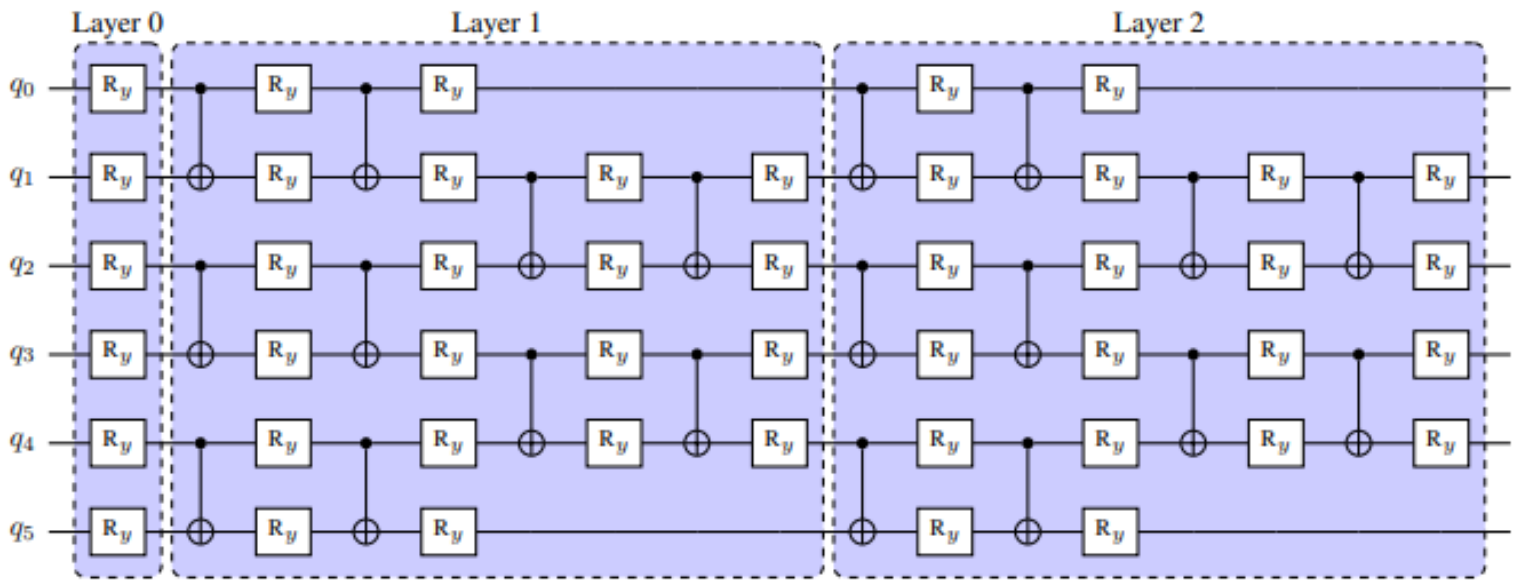
Random deep circuit $\Rightarrow$ gradient vanishes exponentially in n:

$$\mathbb{E}[\partial_k E] = 0, \qquad \mathrm{Var}[\partial_k E] \sim \mathcal{O}\left(\frac{1}{2^n}\right)$$

- Landscape flattens as n, depth grow
- Need $\sim 2^n$ shots to find a direction
- Optimization intractable

L-VQE: build the ansatz layer by layer

Idea: grow the ansatz from a shallow, trainable start, never initialize deep at random.

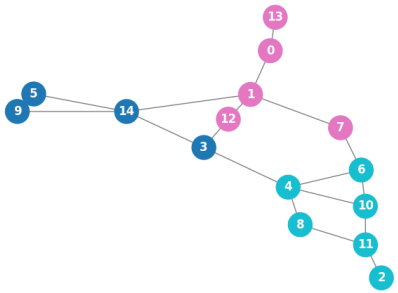


1. **Start shallow:** one R_y/qubit
2. **Partial optimize** (k_0 iters, not to convergence)
3. **Add a layer** at identity: $R_y(0) = I,\ \mathrm{CNOT}^2 = I$
4. **Re-optimize all** from this warm start

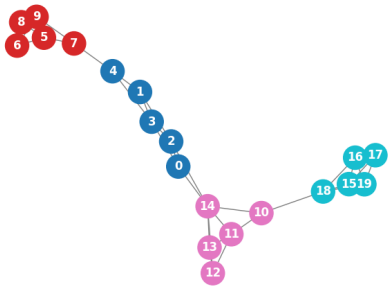
Overall Results: Reproduction on Multiple Graph Types

Graph type	Mean approx. ratio	Mean decoded ratio	Best decoded ratio
Erdős–Rényi ($n = 15$)	0.92	0.93	1.01
Gaussian ($n = 20$)	0.91	0.94	1.00
Windmill ($n = 17$)	0.87	0.88	1.00
Caveman ($n = 20$)	0.85	0.89	1.00

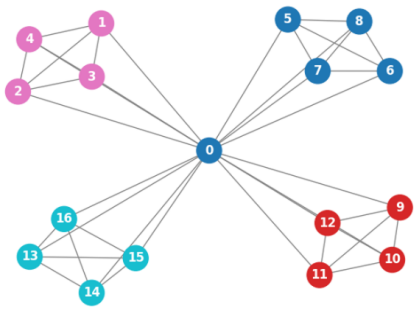
COBYLA, full precision energy, 200 iters/layer + 500 final, 5 seeds each



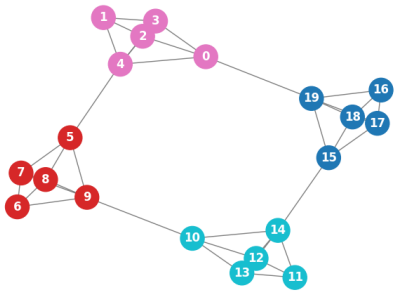
Erdős–Rényi, $\rho = 1.01$



Gaussian, $\rho = 1.00$

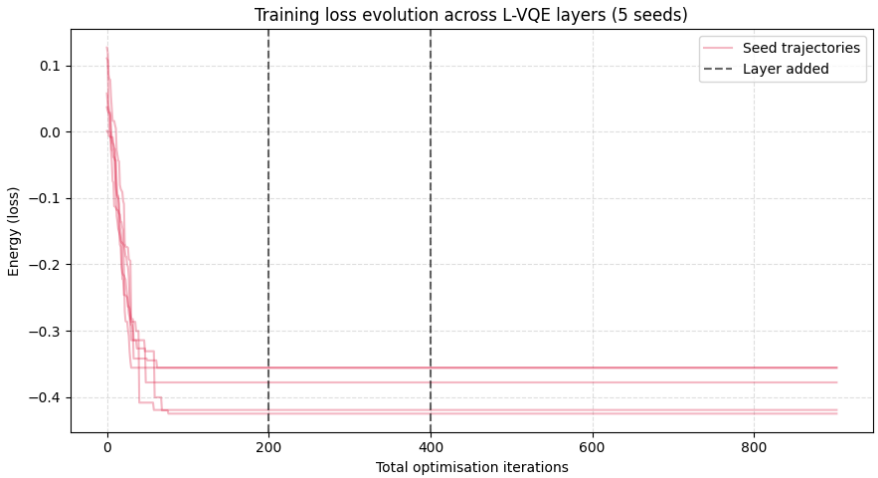


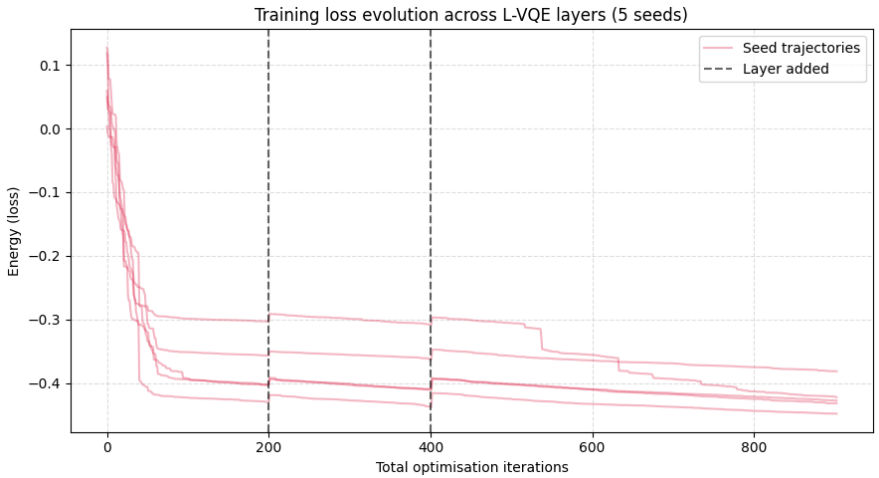
Windmill, $\rho = 1.00$



Caveman, $\rho = 1.00$

SMO: Exact vs. Finite Sampling — Loss Evolution



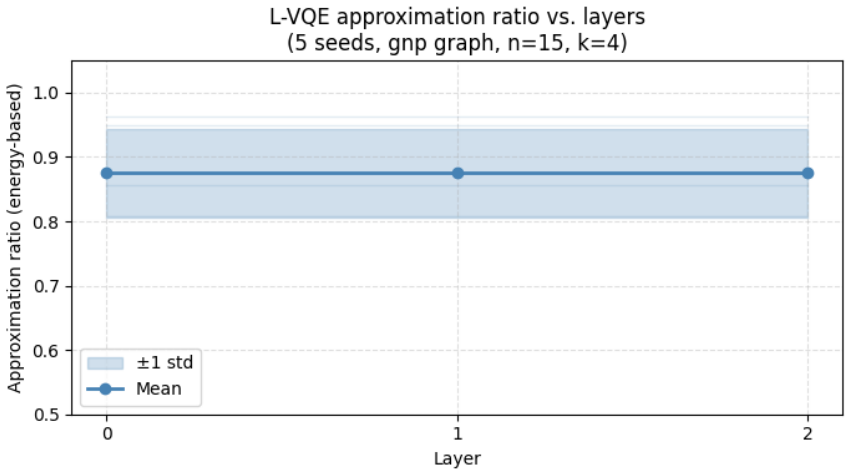


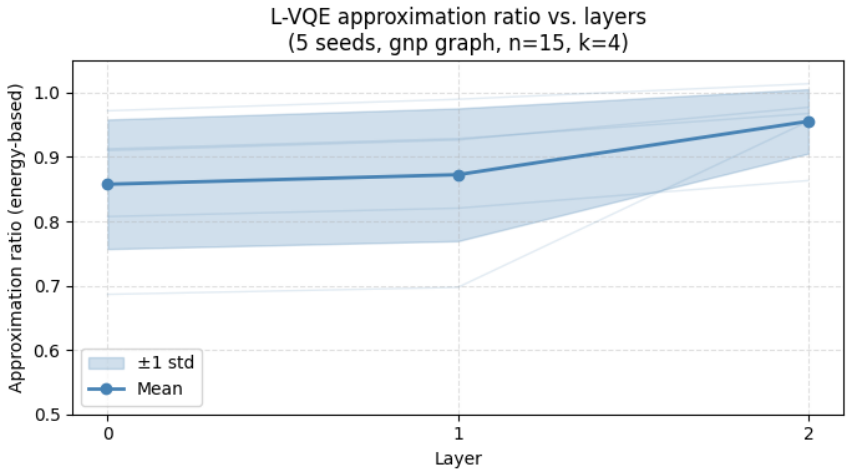
Left: exact expectation

Right: finite sampling ($n_s = 250$)

- Finite sampling slows convergence → more iterations needed to settle
- Sampling noise introduces visible jumps when new layers are added

SMO: Exact vs. Finite Sampling — Approximation Ratio

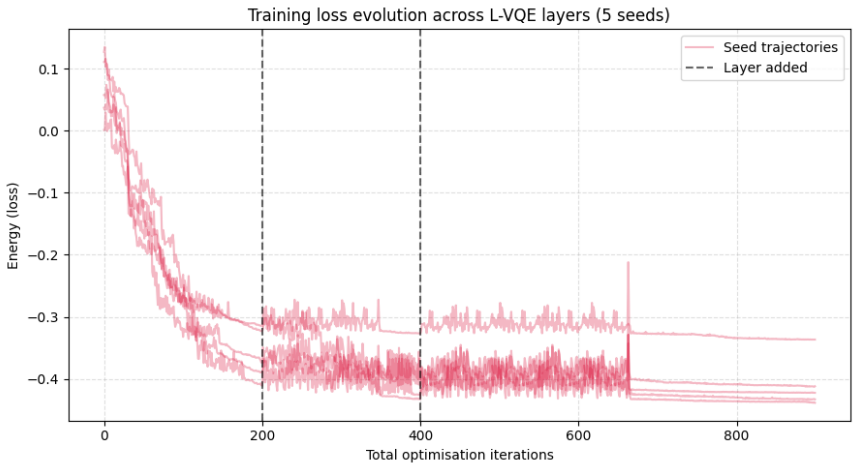


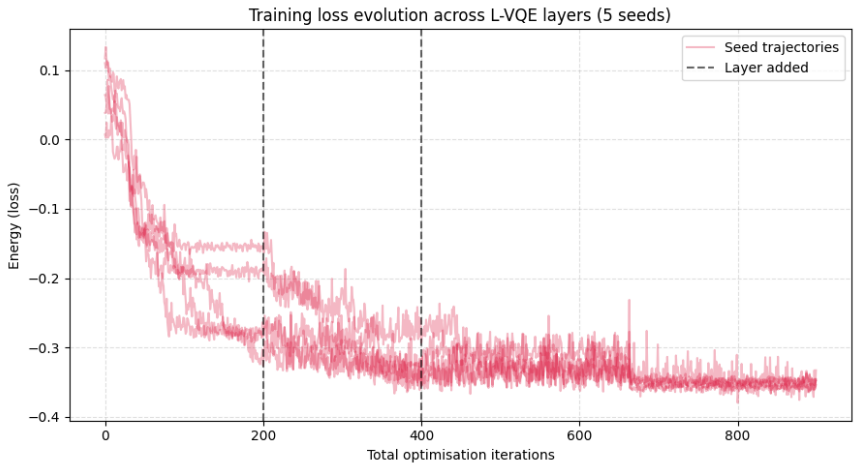


Left: exact expectation **Right:** finite sampling ($n_s = 250$)

- Sampling noise reduces the variance throughout the experiment.
- Energy-based mean approx. ratio is not directly comparable between regimes
- Exact: mean decoded approx. ratio is ≈ 0.87 and best decoded 0.96
- Sampling: mean decoded approx. ratio 0.88 and best decoded 0.95

COBYLA: Exact vs. Finite Sampling — Loss Evolution

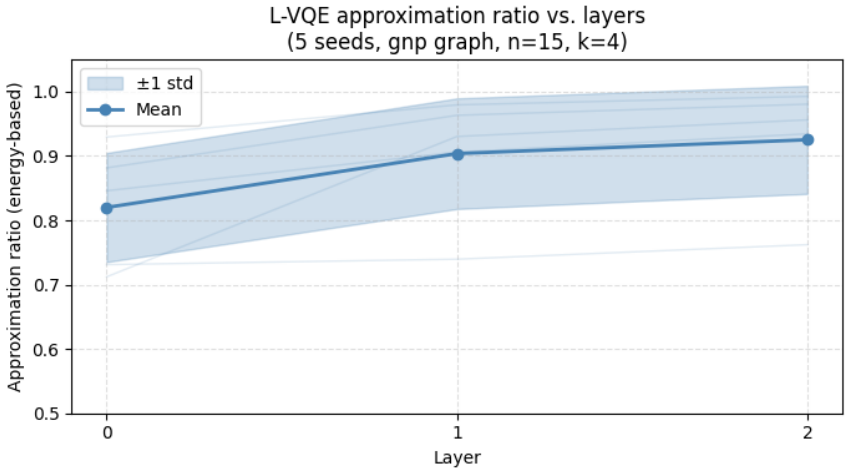


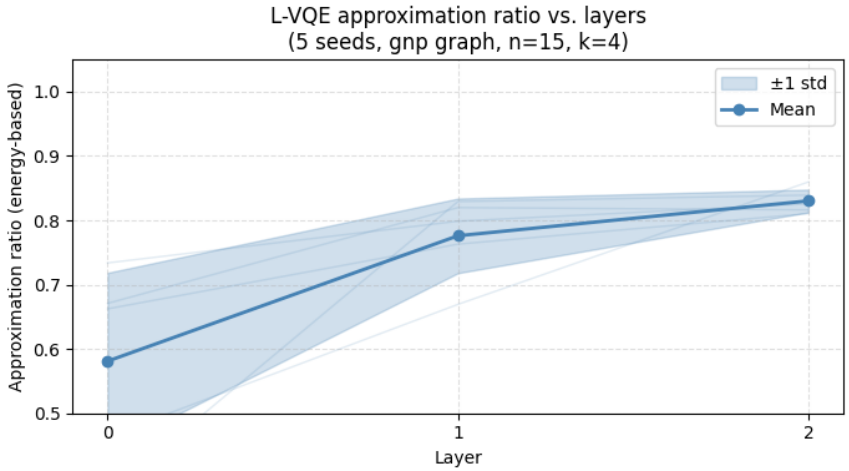


Left: exact expectation **Right:** finite sampling ($n_s = 250$)

- Finite sampling greatly increases noise, making convergence slower and harder harder to stabilize
- Effect is far more pronounced than for SMO, as COBYLA was already noisy.

COBYLA: Exact vs. Finite Sampling — Approximation Ratio

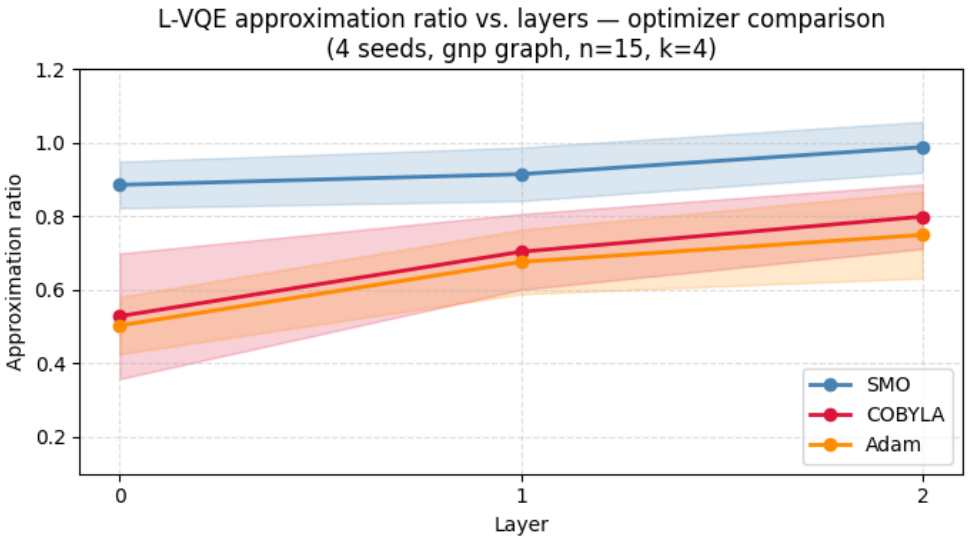


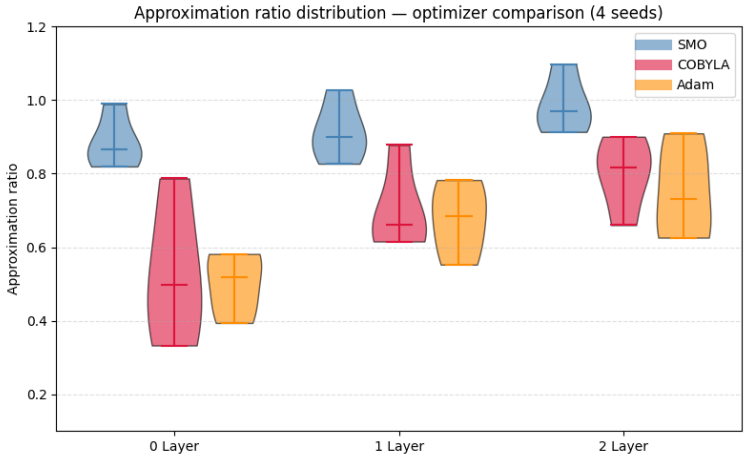


Left: exact expectation **Right:** finite sampling ($n_s = 250$)

- Sampling noise reduces the variance throughout the experiment.
- Energy-based mean approx. ratio is not directly comparable between regimes, only the decoded energy is
- Decoded mean approx.ratio is 0.93 (exact) vs 0.91 (sampling)
- Decoded best approx. ratio is 1.00 (exact) vs 0.99 (sampling)

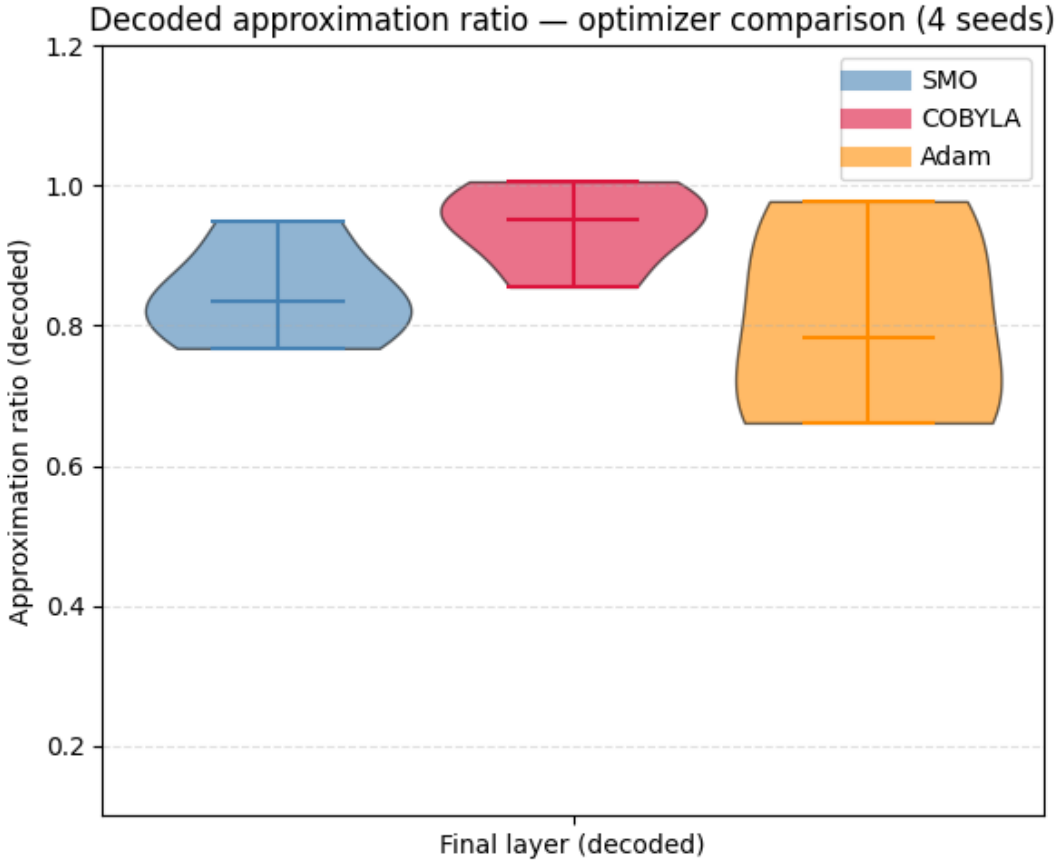
Optimizer Comparison: Approximation Ratio (Energy)





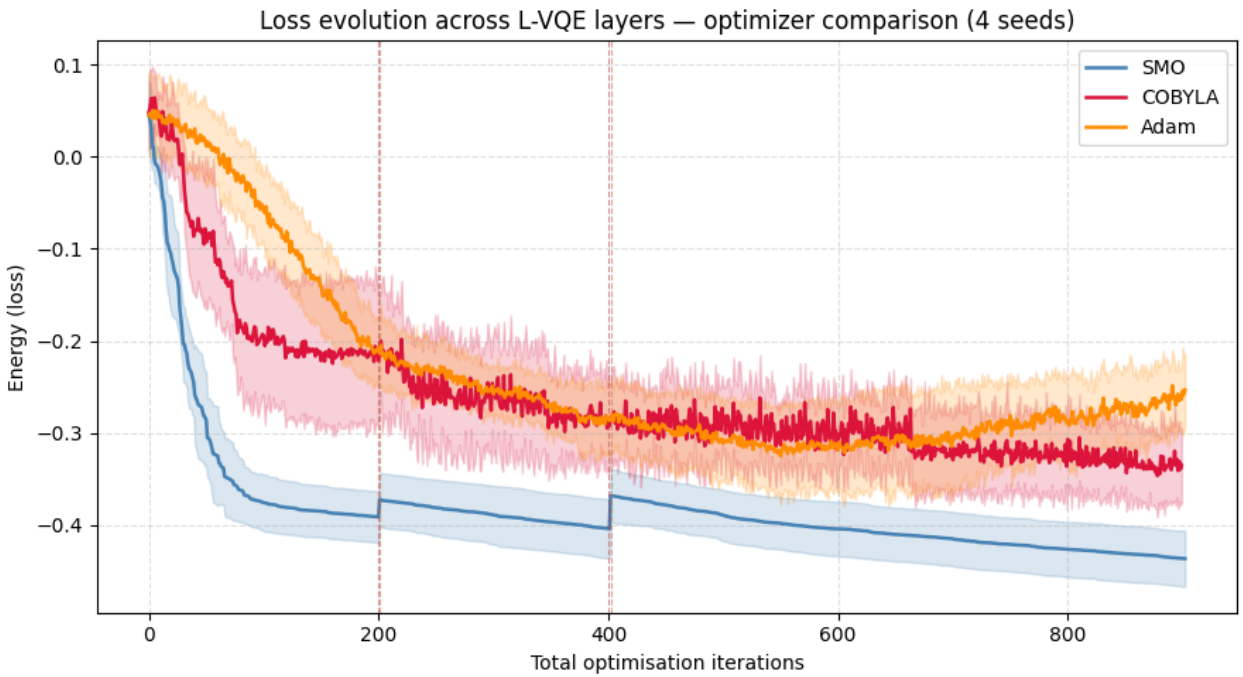
- SMO consistently outperforms COBYLA and Adam on the energy-based ratio
- COBYLA reduces its standard deviation throughout the run.

Optimizer Comparison: Decoded Approximation Ratio



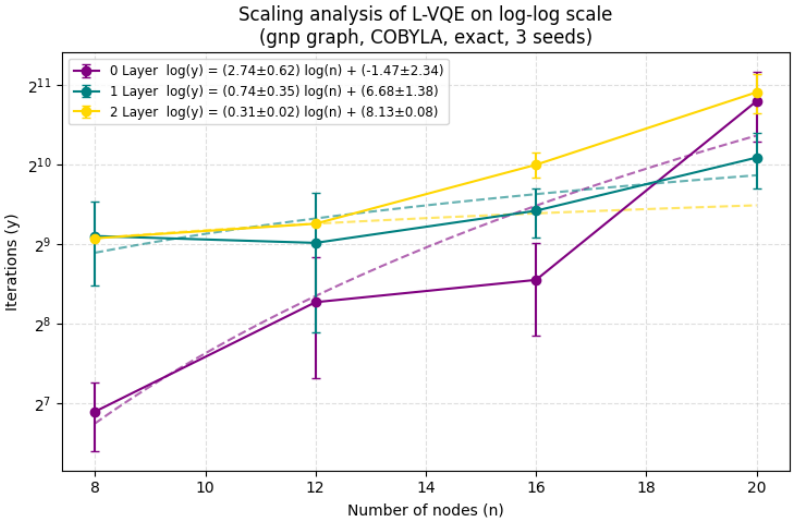
- Decoded picture changes the ranking as COBYLA matches or exceeds SMO
- Adam remains the most variable and lowest on average

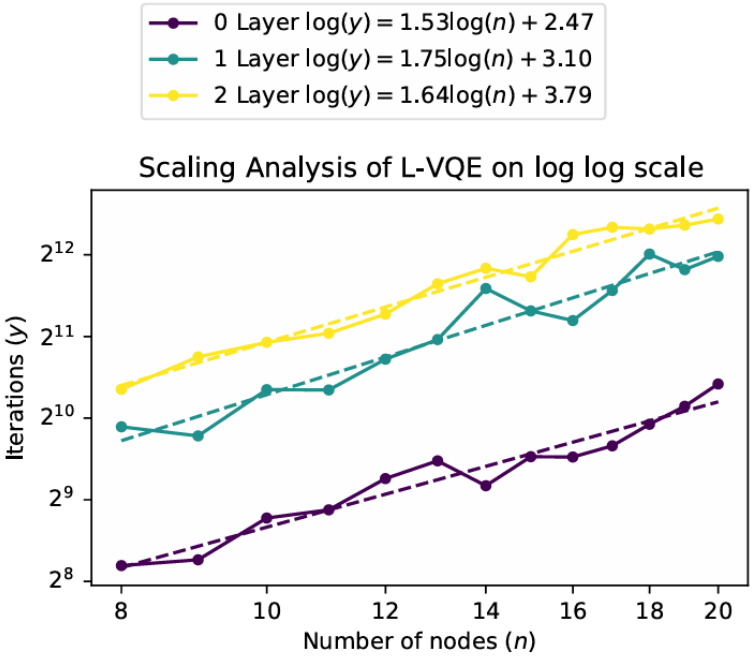
Optimizer Comparison: Loss Evolution



- SMO converges fastest and with the smoothest trajectory
- COBYLA and Adam remain noisy throughout
- Despite this, COBYLA's decoded result remains the best and most stable.

Scaling Analysis: Iterations to Convergence vs. Graph Size





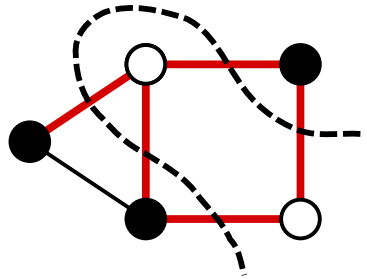
Left: our reproduction

Right: paper's Fig. 6

- the paper does not define graph type, instance count or convergence criterion, so it is hard to actually reproduce it.
- We tuned ε to match the paper's order of magnitude and measure convergence only in the final layer (sa the paper seems to have done)
- For 1 and 2 layers, earlier layers (200 iters) already do most of the optimization → final-layer convergence is fast and largely n-independent

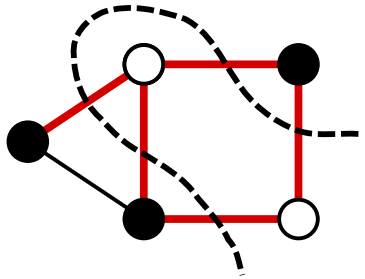
MaxCut: Problem and Encoding

Problem: Given $G = (V, E)$, partition V into S, T maximizing the number of cut edges. NP-hard; natural Ising form.



MaxCut: Problem and Encoding

Problem: Given $G = (V, E)$, partition V into S, T maximizing the number of cut edges. NP-hard; natural Ising form.



Encoding: (constant term dropped)

$$H = -\frac{1}{2} \sum_{(u,v)\in E} Z_u Z_v, \quad \text{cut} = \langle H \rangle + \frac{|E|}{2}$$

Symmetry reduction: Global bit flip leaves the cut invariant
$\Rightarrow$ fix node 0: $n_{\text{qubits}} = N - 1$
(Amaro et al. 2022)

Reference values:

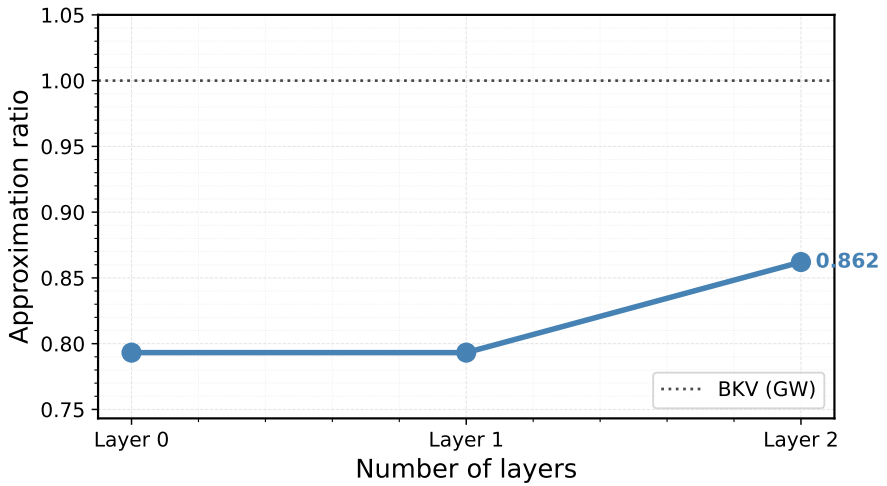
- $N \leq 20$: brute force
- $N > 20$: GW (Goemans and Williamson 1995)
  $\Rightarrow \alpha \geq 0.879$ in poly time

Instances: random connected 3-regular graphs, varying N

MaxCut: Results — Simple Experiment, Exact Expectation

Experimental setup: $N = 42$, 2 layers, SMO optimizer, exact expectation

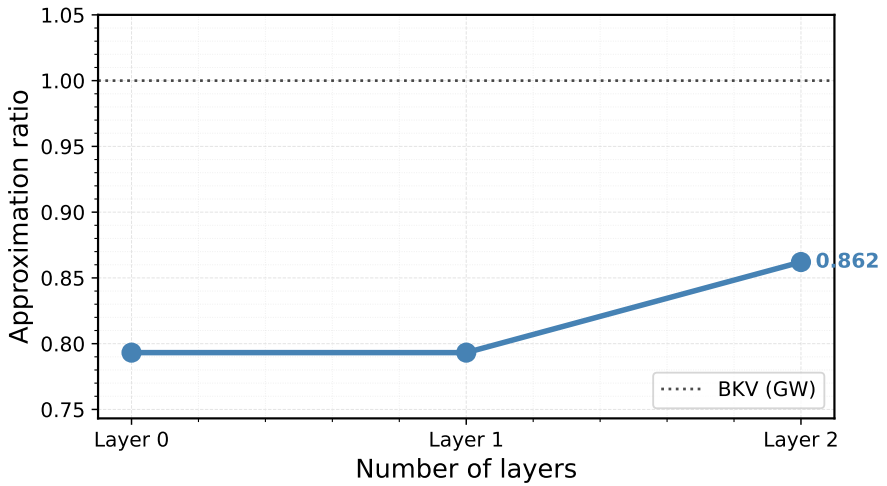
- Best known (GW): $\mathrm{cut} = 58$
- L-VQE: obtained $\mathrm{cut} = 50$
  $\Rightarrow \alpha = 0.862$
- Sampled state concentrates on a single assignment



MaxCut: Results — Simple Experiment, Exact Expectation

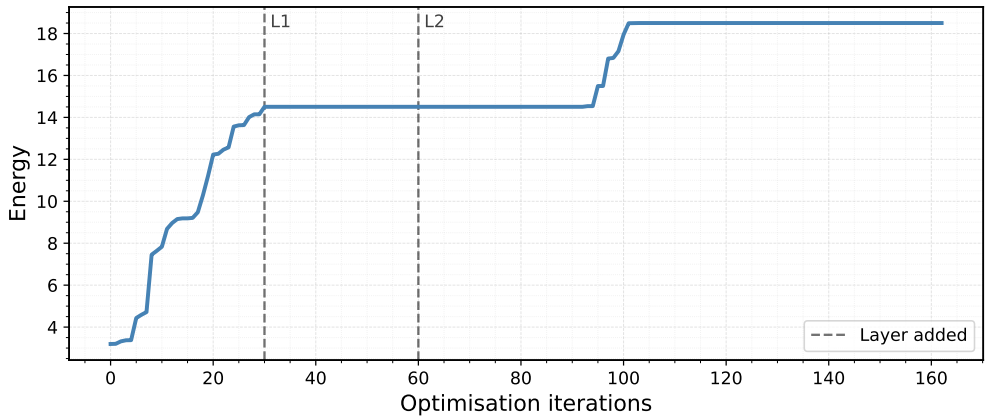
Experimental setup: $N = 42$, 2 layers, SMO optimizer, exact expectation

- Best known (GW): $\mathrm{cut} = 58$
- L-VQE: obtained $\mathrm{cut} = 50$
  $\Rightarrow \alpha = 0.862$
- Sampled state concentrates on a single assignment



Warm start: energy increases monotonically, *no reset* when layers are added

Was the local minimum escaped thanks *to the addition of a new layer? Or was it* caused *by it?*



MaxCut: Results — Finite Sampling: L-VQE vs. Base VQE

Experimental setup:

- *Base VQE* = fixed-ansatz VQE baseline, as used in Sec. VI-B of Liu et al. (2022)
- 10 random 3-regular instances, $N = 52$
- For each $n_{\text{layers}} \in \{0, 1, 2, 3\}$ and for each model, take the best-of-3 runs per instance, then mean ± SEM over the 10 instances.
- Equal total budget:

$$n_{\text{layers}} \cdot k_{\text{layer}} + k_{\text{final}} = 500 \text{ SMO iterations} \quad (k_{\text{layer}} = 50)$$

- *Realistic* regime: finite sampling, $n_{\text{samples}} = 50$
- Parallelized over instances; tracked with MLflow

MaxCut: Results — Finite Sampling: L-VQE vs. Base VQE

Experimental setup: $N = 52$, 10 random 3-regular instances, best of 3 runs, mean ± SEM; equal budget of 500 SMO iterations; finite sampling, $n_{\text{samples}} = 50$

Observations:

- Clear separation from $n_{\text{layers}} = 2$: L-VQE beats Base VQE[a]
- Base VQE never recovers the 0-layer (product-state) baseline
- L-VQE shows smaller SEM

[a] At $n_{\text{layers}} = 2$, L-VQE performs better on $9/10$ instances. A paired Wilcoxon signed-rank test supports the claim with $p = 0.005$.

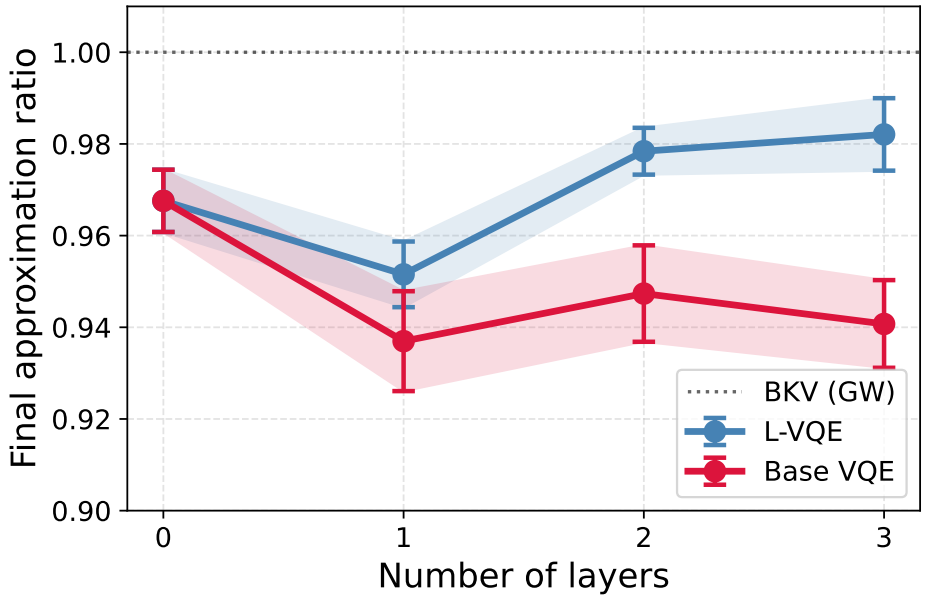


Figure: Final approximation ratio vs. number of ansatz layers. Markers: mean over the 10 per-instance best-of-3 values; bands: ± SEM.

MaxCut: Results — Finite Sampling: L-VQE vs. Base VQE

Experimental setup: $N = 52$, 10 random 3-regular instances, best of 3 runs, mean ± SEM; equal budget of 500 SMO iterations; finite sampling, $n_{\text{samples}} = 50$

Observations:

- Clear separation from $n_{\text{layers}} = 2$: L-VQE beats Base VQE[a]
- Base VQE never recovers the 0-layer (product-state) baseline
- L-VQE shows smaller SEM

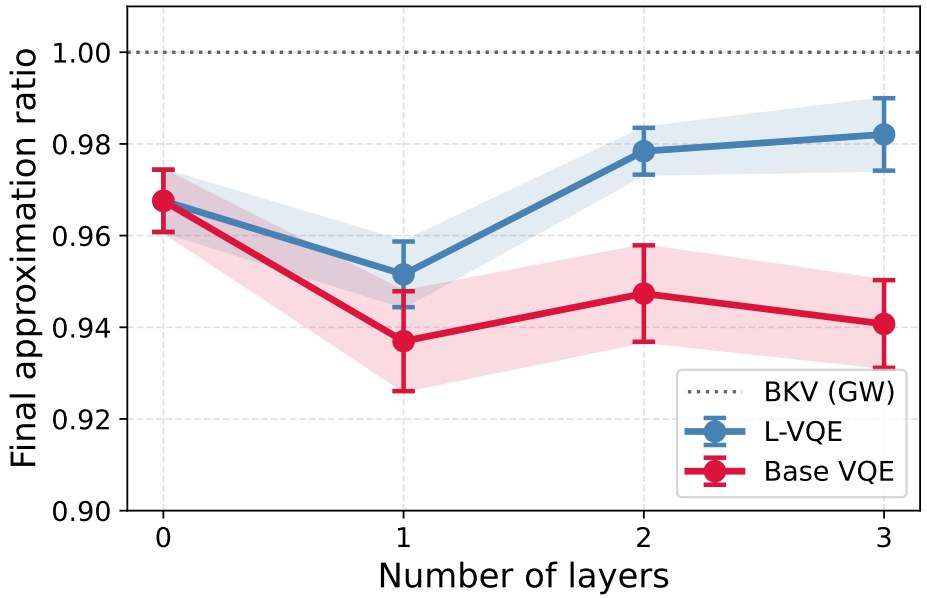


Figure: Final approximation ratio vs. number of ansatz layers. Markers: mean over the 10 per-instance best-of-3 values; bands: ± SEM.

[a] At $n_{\text{layers}} = 2$, L-VQE performs better on $9/10$ instances. A paired Wilcoxon signed-rank test supports the claim with $p = 0.005$.

⇒ consistent with the paper's observation: L-VQE is more robust to finite sampling errors

Extension 2: L-VQE on IBM Quantum hardware

Problem

- Planted partition: 2×5 nodes (21 edges)
- 1 bit/node $\Rightarrow$ 10 qubits; 46-term ZZ Hamiltonian
- $Q_{\text{best}} = 0.4524$

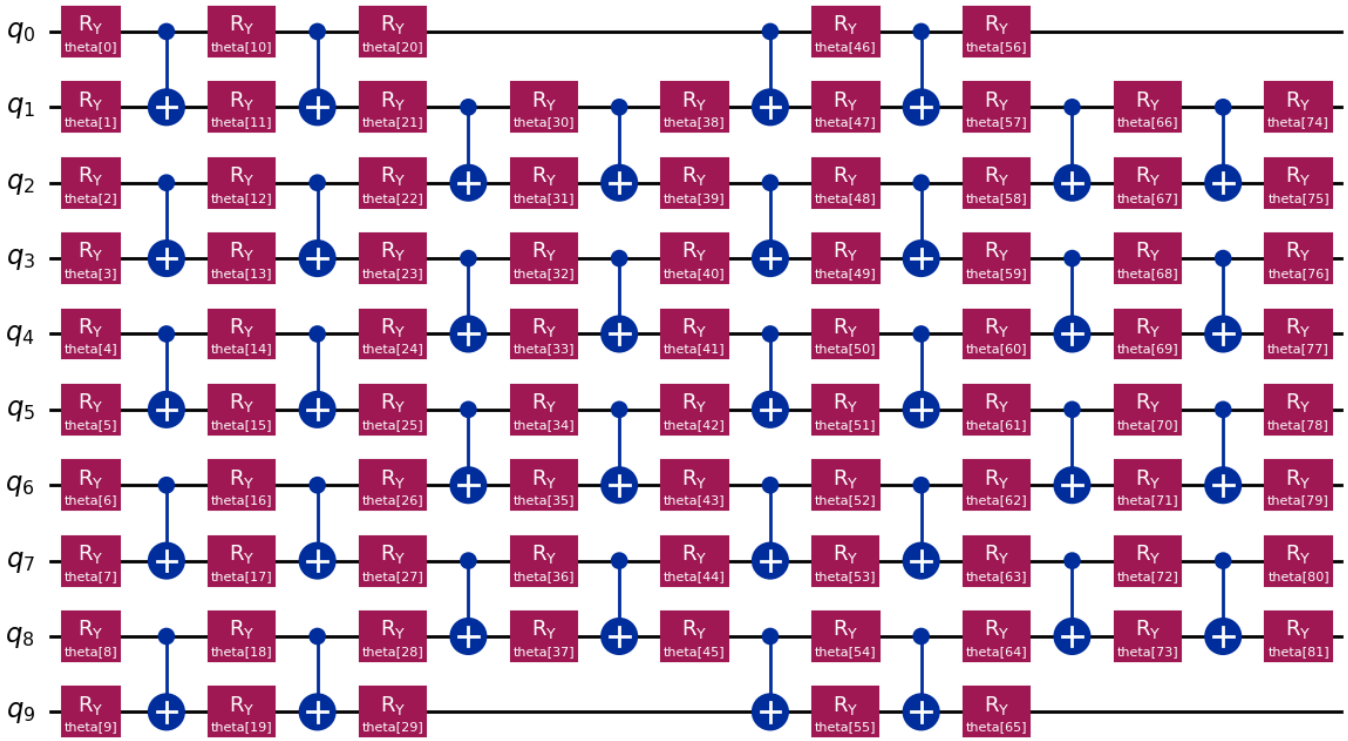
L-VQE

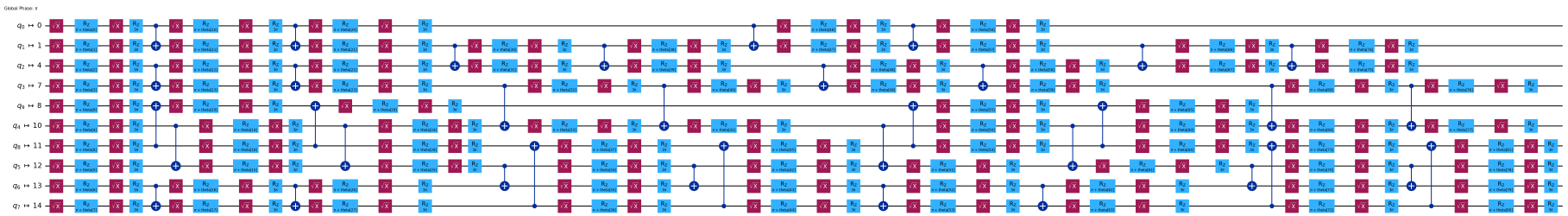
- COBYLA, 2 layers, 82 params
- $100/100/300$ iters, 1024 shots

Three noise scenarios:

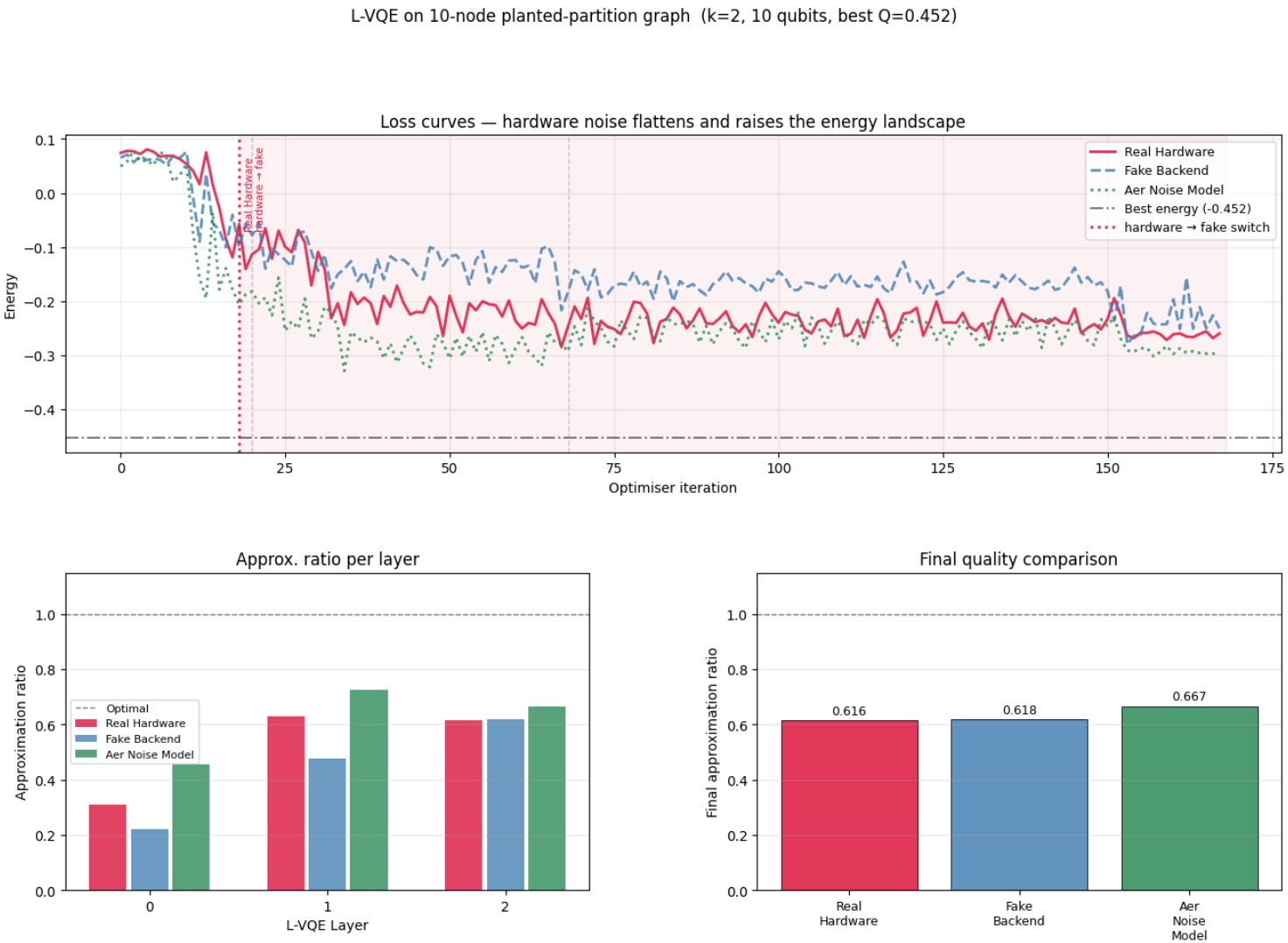
- Real Hardware: `ibm_marrakesh`
- Fake Backend: `FakeGuadalupeV2` (Pauli-error model)
- Aer: T_1/T_2 + gate + readout

From Abstract Circuit to Actual Device

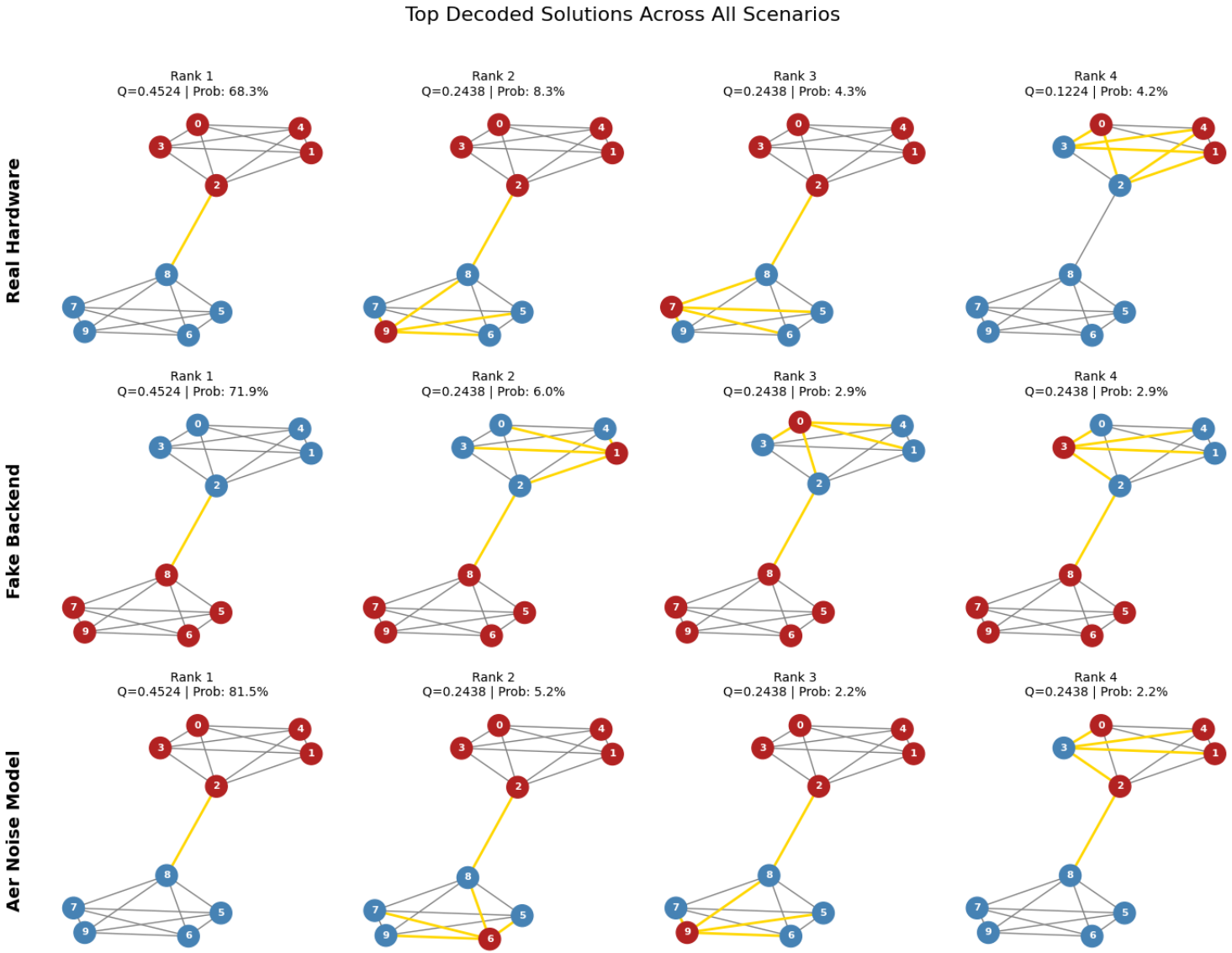




Results: Training



Results: Decoding



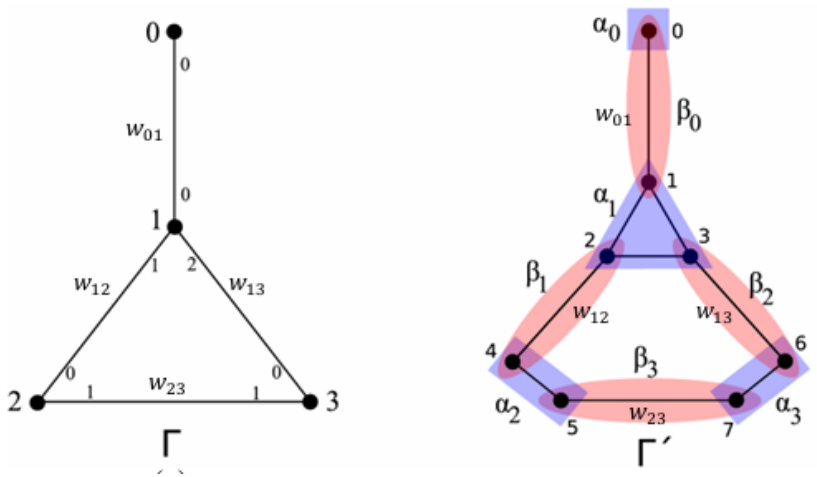
Bonus: Weighted Staggered Quantum Walk

Clique-insertion: replace each vertex v_i by a clique over its neighbours in extended graph G'.

Tessellation states:

$$|\alpha_i\rangle = \frac{1}{\sqrt{\sum_j w_{ij}}} \sum_j \sqrt{w_{ij}} \, |v_{ij}\rangle$$

$$|\beta_{ij}\rangle = \frac{|v_{ij}\rangle + |v_{ji}\rangle}{\sqrt{2}}$$



Clique-insertion graph H

Reflection operators:

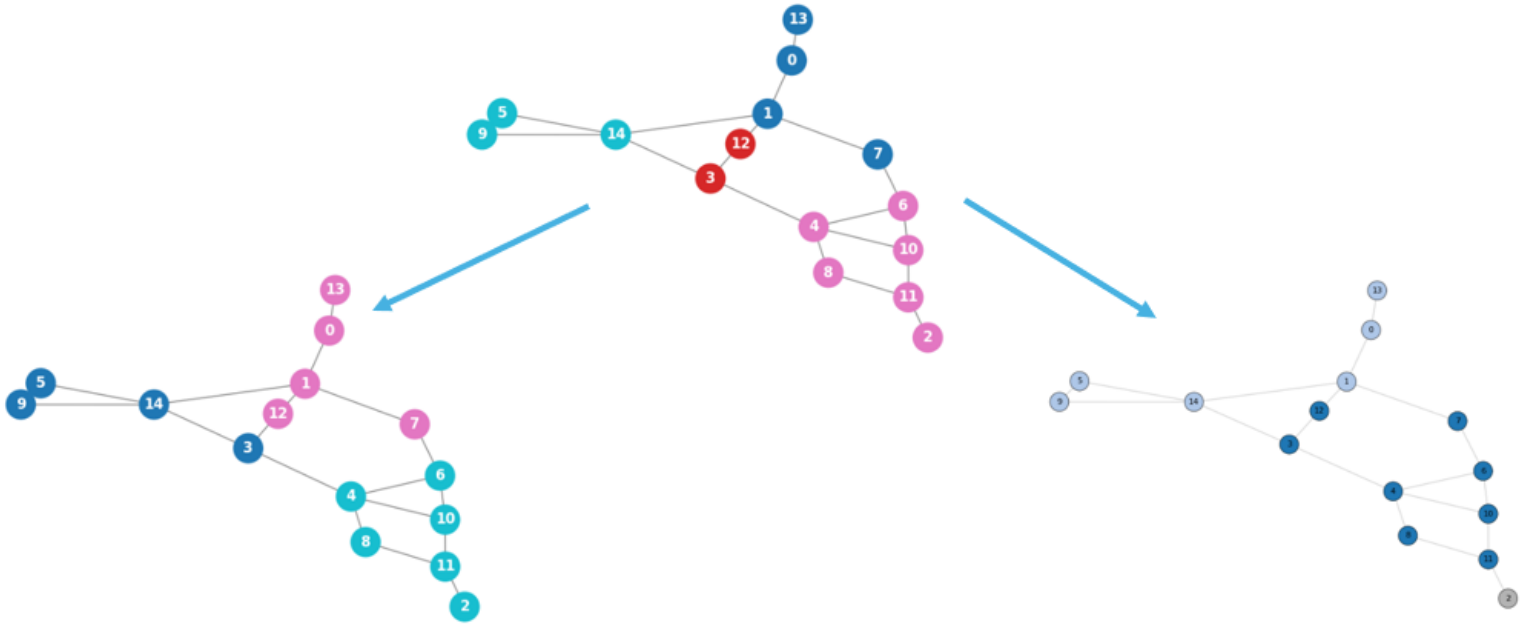
$$W_\alpha = I - 2\sum_i |\alpha_i\rangle\langle\alpha_i|$$

$$W_\beta = I - 2\sum_{ij} |\beta_{ij}\rangle\langle\beta_{ij}|$$

Walk operator: $U = W_\beta W_\alpha$

Community detection: evolve until time-averaged edge probability Π^T converges and apply classical procedure to identify communties.

Random Graph: L-VQE vs Quantum Walk



L-VQE $\rho = 1.01$

750 iterations ≈ 1500 cost evaluations

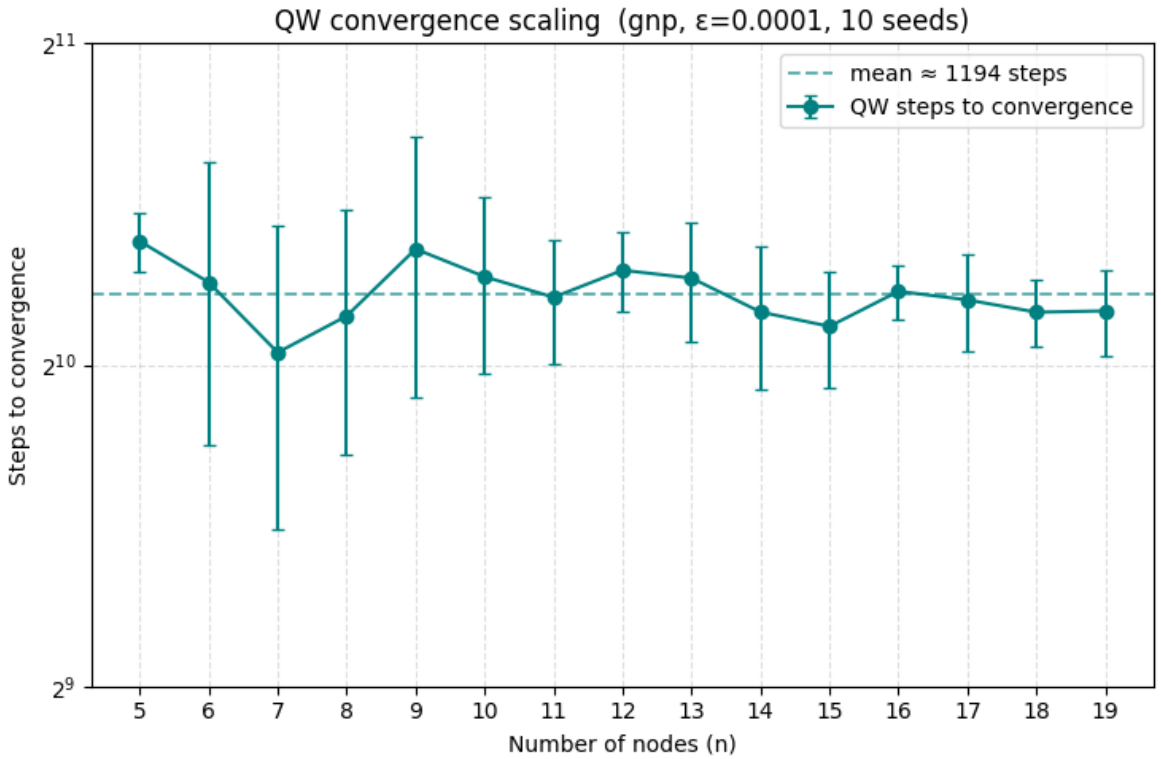
L-VQE still recovers a good partition, according to the metric of Modularity

Quantum Walk $\rho = 0.674$

1200 steps to convergence of Π

Limiting probability carries insufficient signal for the threshold procedure

Quantum Walk: Convergence Scaling



Steps to convergence is roughly constant (≈ 1200) across graph sizes.

High variance reflects the random graph structure, as we are varying the graph for each seed.

Conclusions

- We reproduced the paper's results, extending the analysis on finite sampling and an additional Adam-based optimizer.
- We applied L-VQE to MaxCut and benchmarked it against Base VQE, confirming the paper's noise-robustness claim on a new problem.
- We implemented L-VQE on real IBM Quantum hardware, comparing against a fake backend and an Aer noise model.
- As a bonus, we compared L-VQE against a quantum-native heuristic, the Staggered Quantum Walk, for community detection.

Thank you for listening!

Questions?

Scan the QR code to view the repository

References I

Amaro, D., C. Modica, M. Rosenkranz, M. Fiorentini, M. Benedetti, and M. Lubasch (2022). "Filtering variational quantum algorithms for combinatorial optimization". In: *Quantum Science and Technology* 7.1, p. 015021. DOI: 10.1088/2058-9565/ac3e54.

Goemans, M. X. and D. P. Williamson (Nov. 1995). "Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming". en. In: *Journal of the ACM* 42.6, pp. 1115–1145. ISSN: 0004-5411, 1557-735X. DOI: 10.1145/227683.227684.

Liu, X., A. Angone, R. Shaydulin, I. Safro, Y. Alexeev, and L. Cincio (2022). "Layer VQE: A Variational Approach for Combinatorial Optimization on Noisy Quantum Computers". en. In: *IEEE Transactions on Quantum Engineering* 3, pp. 1–20. ISSN: 2689-1808. DOI: 10.1109/TQE.2021.3140190.

McClean, J. R., S. Boixo, V. N. Smelyanskiy, R. Babbush, and H. Neven (2018). "Barren plateaus in quantum neural network training landscapes". In: *Nature Communications* 9.1, p. 4812. ISSN: 2041-1723. DOI: 10.1038/s41467-018-07090-4.

Rahaman, M. S. and S. Dutta (2026). *Community detection in network using Szegedy quantum walk*. arXiv: 2601.21152 [quant-ph].

References II


Schwägerl, T., Y. Chai, T. Hartung, K. Jansen, and S. Kühn (June 2026). "Benchmarking variational quantum algorithms for combinatorial optimization in practice". en. In: *Quantum Machine Intelligence* 8.1, p. 26. ISSN: 2524-4906, 2524-4914. DOI: 10.1007/s42484-026-00355-y.

Backup: What we implemented

Ansatz (as in the paper)

- Layer 0: one R_y per qubit, random init in $[0, 2\pi]$
- Layer $l > 0$: brickwork CNOT + R_y blocks, $4n_q - 4$ new parameters
- New layers zero-initialized $\Rightarrow$ identity at insertion: warm start

L-VQE schedule: optimize layer 0 for k_{layer} iters; add a layer, re-optimize; final layer gets k_{final} iters

Simulator: quimb tensor networks (MPS circuits)

- Exact mode: $\langle\psi|H|\psi\rangle$
- Finite sampling: energy estimated from sampled bitstrings

Optimizers:

- COBYLA (derivative-free)
- SMO: analytic per-parameter optimum, 3 evals/step
- Adam + parameter shift (bonus, not in paper)

Backup: COBYLA Optimizer

Constrained Optimization BY Linear Approximations

Maintains a set of $n_\theta + 1$ points in parameter space.

1. Fit a linear model $f(\theta) \approx c + g^\top \theta$ through the stored vertices
2. Minimize the linear model inside a trust region of radius ρ around the current best point
3. Evaluate the trial point. If it improves → accept and update the simplex. If not → shrink ρ
4. ρ decreases monotonically from `rhobeg` to `rhoend`

Key properties:

- Updates all parameters jointly
- Simplex initialization costs $n_\theta + 1$ evaluations before the main loop starts
- Total evaluations $\approx n_\theta + 2 \times$ `maxiter`
- No gradient, no momentum ⇒ sensitive to evaluation noise

Backup: SMO Optimizer

Sequential Minimal Optimization

Makes use of the fact that $E(\theta_k)$ is sinusoidal due to R_y:

$$E(\theta_k) = A\sin(\theta_k + \varphi) + C$$

Computes the minimum of the function by, first, determining the three unknowns: A, φ and C.

1. Pick parameter $k = \texttt{iteration} \bmod n_\theta$
2. Evaluate $E(\theta_k + \pi/2)$ and $E(\theta_k - \pi/2)$ (reuse $E(\theta_k)$ from previous step)
3. Solve for A, φ, C and jump directly to the analytic minimum θ_k^*
4. Set $E(\theta_k^*) = C - \frac{1}{2}\sqrt{(f_- - f_+)^2 + (2f_0 - f_+ - f_-)^2}$

Key properties:

- Each step is guaranteed to not increase the objective
- 2 new evaluations per iteration → total $\approx 2 \times \texttt{maxiter}$
- Updates one parameter at each iteration.

Backup: Adam Optimizer

Adaptive Moment Estimation Gradient obtained via the parameter-shift rule:

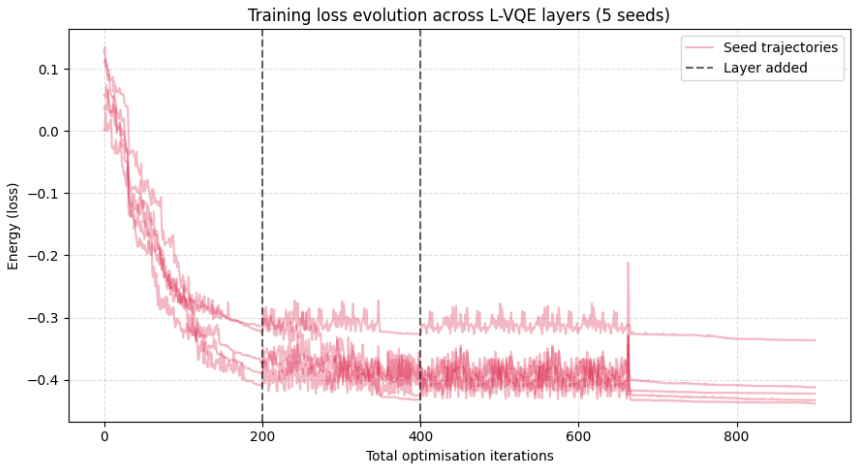
$$\frac{\partial E}{\partial \theta_k} = \frac{E(\theta_k + \pi/2) - E(\theta_k - \pi/2)}{2}$$

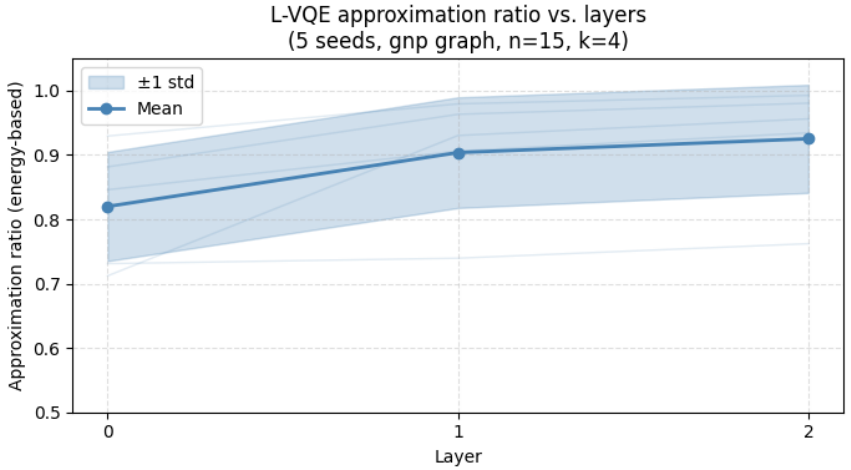
1. Pick parameter k, compute gradient g_k via parameter shift (2 evaluations)
2. Update the first moment (momentum): $m_k \leftarrow \beta_1 m_k + (1 - \beta_1)\, g_k$
3. Update second moment (magnitude): $v_k \leftarrow \beta_2 v_k + (1 - \beta_2)\, g_k^2$
4. Bias-correction: $\hat{m} = m_k/(1 - \beta_1^t), \quad \hat{v} = v_k/(1 - \beta_2^t)$
5. Update: $\theta_k \leftarrow \theta_k - \eta\, \hat{m} \,/\, (\sqrt{\hat{v}} + \varepsilon)$

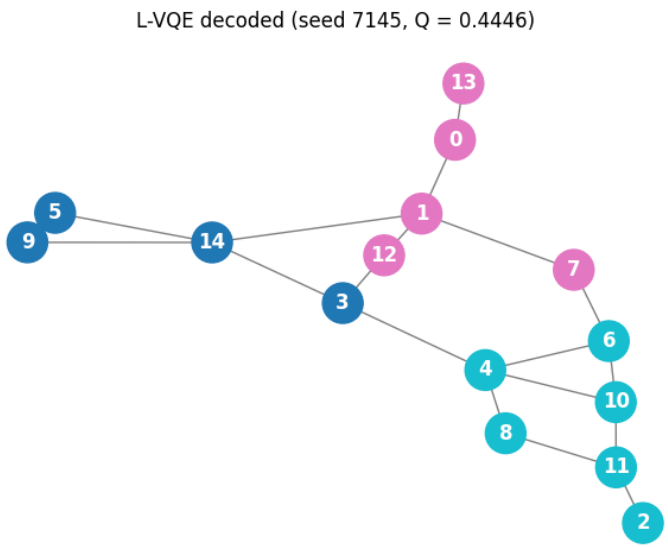
Key properties:

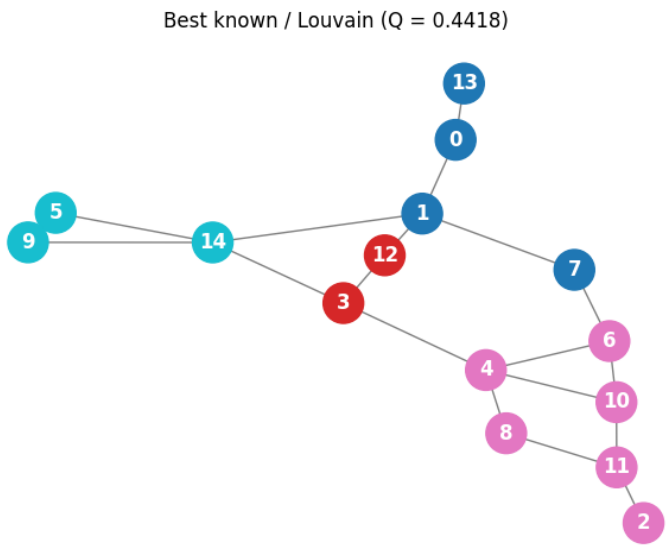
- Momentum smooths noisy gradients but introduces lag when the landscape changes (as it happens with the addition of layers)
- 3 evaluations per iteration (shift + evaluation at updated point)
- Hyperparameter-sensitive $\rightarrow \eta$, β_1, β_2. All affect convergence and performance

Backup: Erdős–Rényi — Full Results

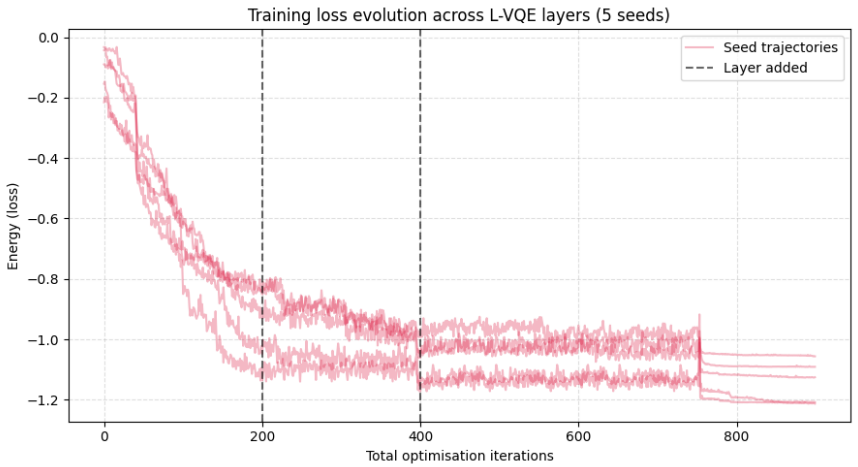


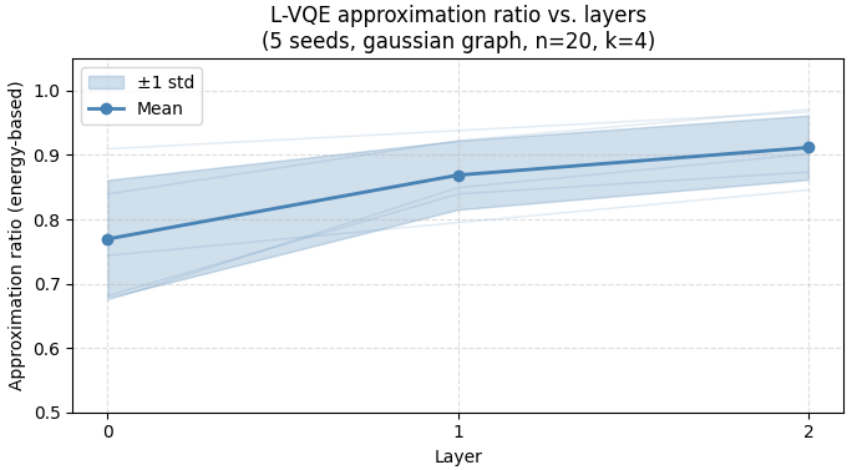


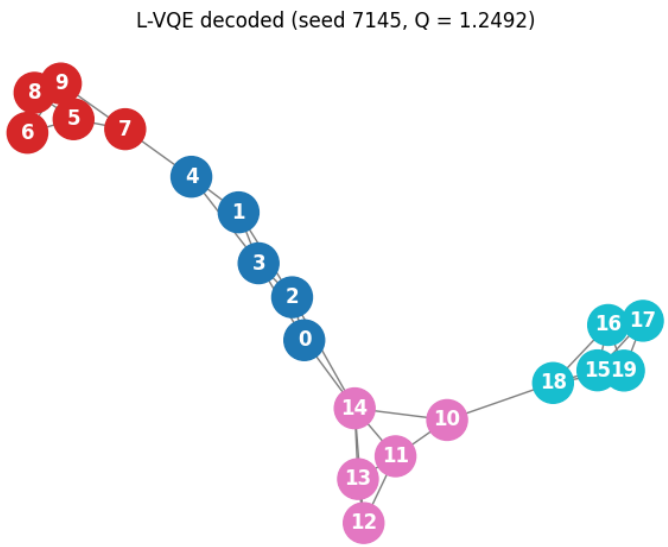


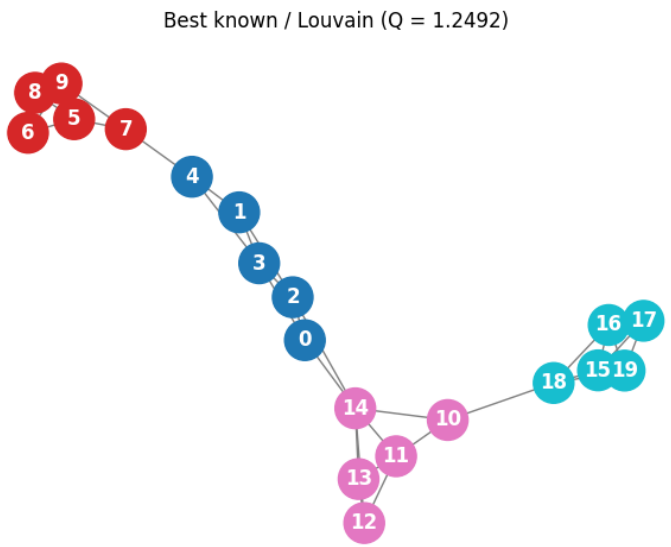


Backup: Gaussian — Full Results

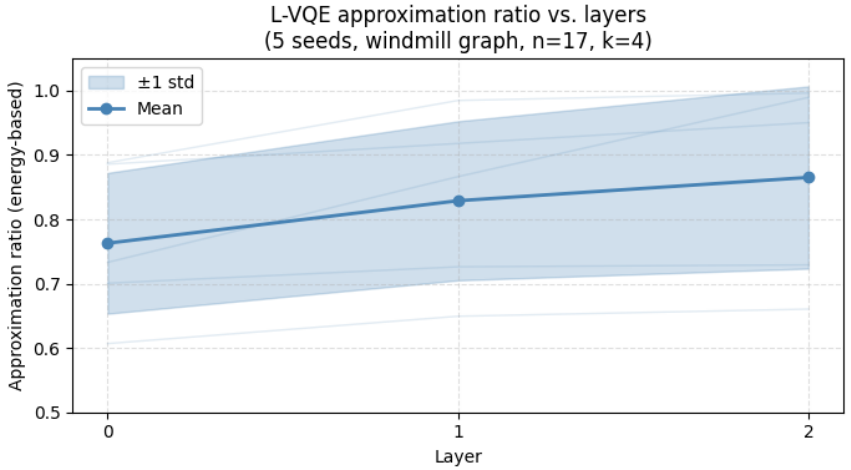


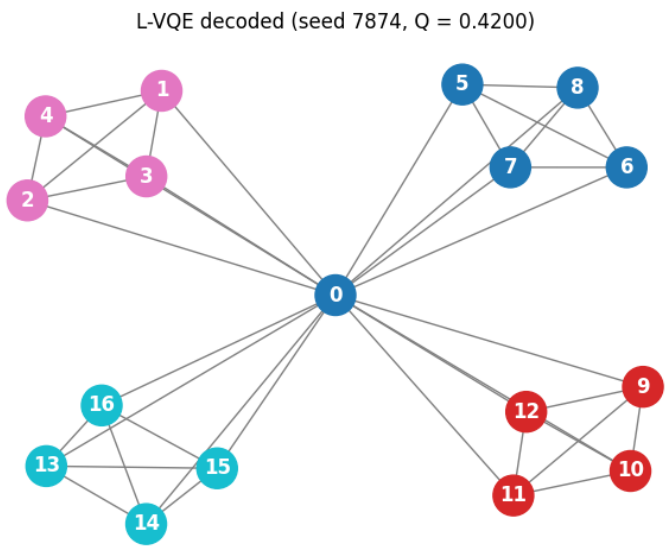


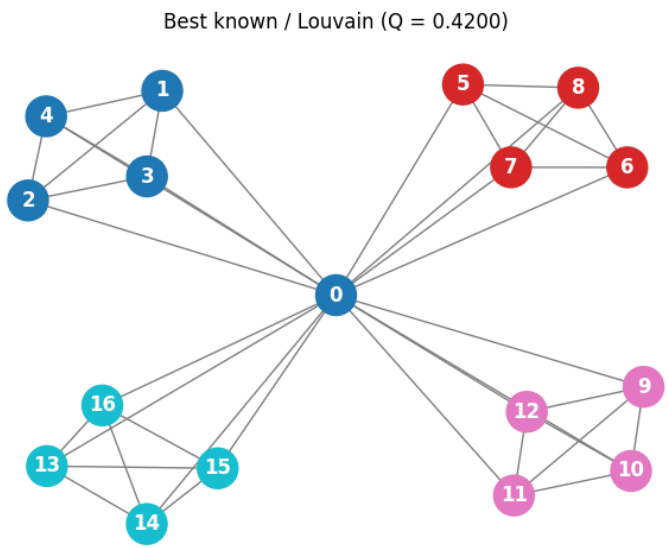




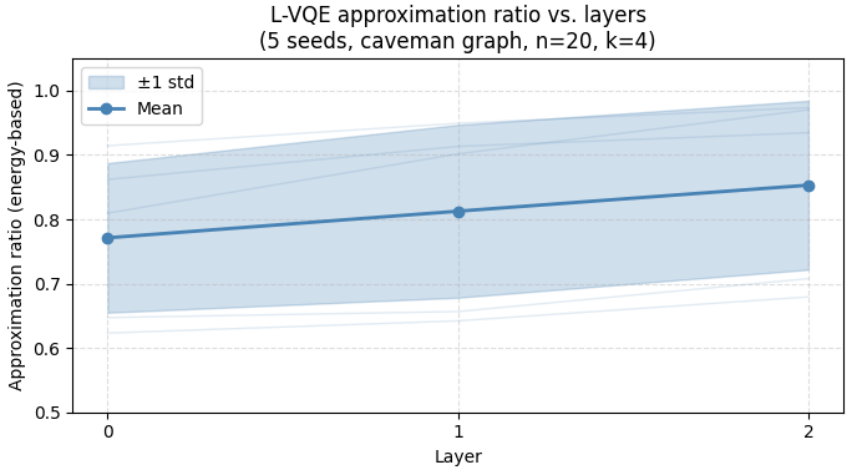
Backup: Windmill — Full Results

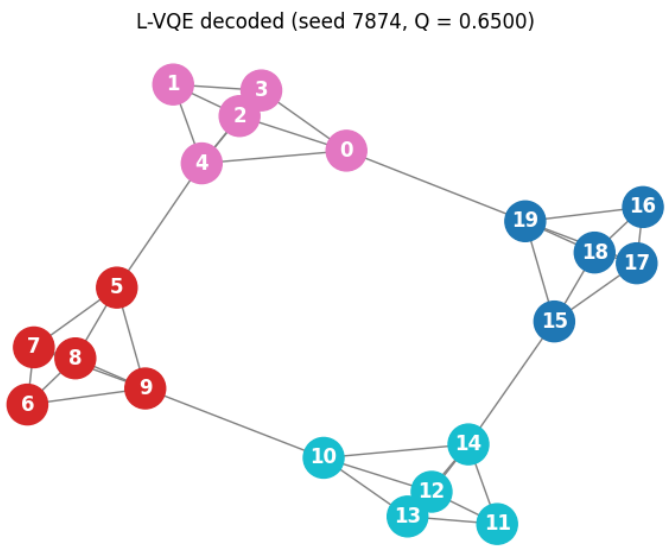


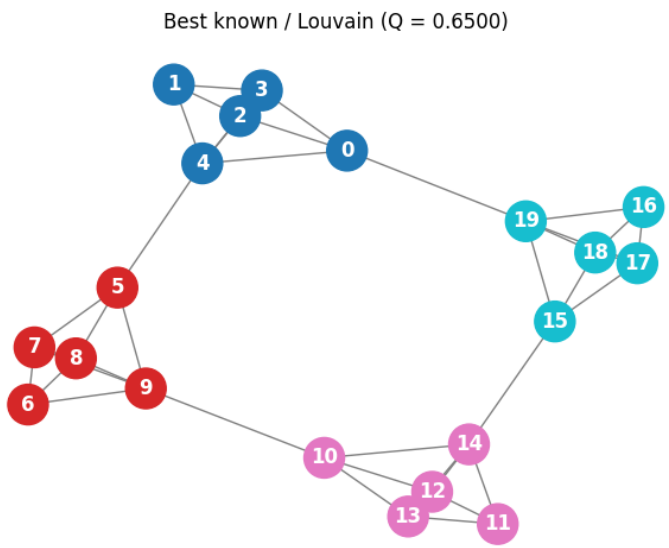




Backup: Caveman — Full Results







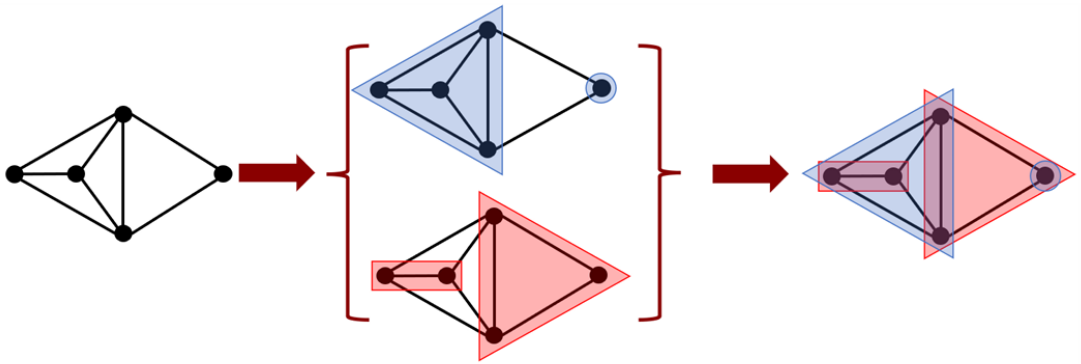
Backup: Staggered Quantum Walk Framework

Tessellation cover

$$\{\mathcal{T}_{\text{red}}, \mathcal{T}_{\text{blue}}\}$$

Polygon state

$$|\alpha_j\rangle = \frac{1}{\sqrt{|\alpha_j|}} \sum_{v \in \alpha_j} |v\rangle$$



Tessellations induce reflections that compose the walk

Operators

$$W_\alpha = I - 2\sum_{j=1}^{p} |\alpha_j\rangle\langle\alpha_j|$$

Evolution:

$$U = \prod_{i=1}^{k} e^{i\theta_i W_i}, \qquad \theta_i = \frac{\pi}{2} \Rightarrow U = \prod_{i=1}^{k} W_i$$

Backup: Weighted Staggered Quantum Walk

Goal: Extend the staggered quantum walk model to weighted graphs.

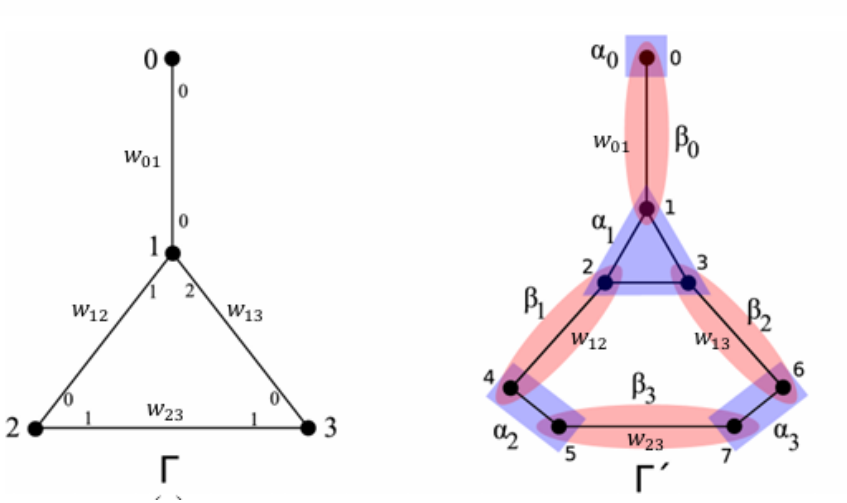
Step 1: Clique-insertion operator

- Replace each vertex v_i by a clique.
- Encode edge weights w_{ij} locally.

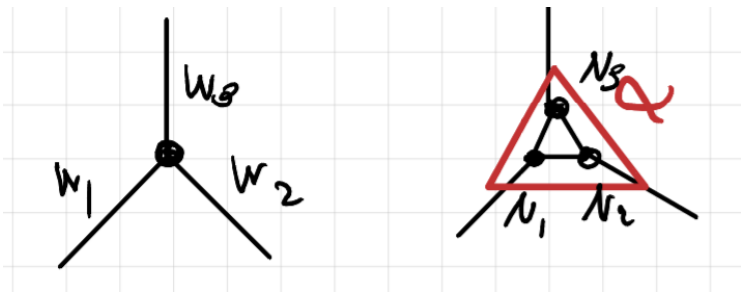
Step 2: Tessellation cover

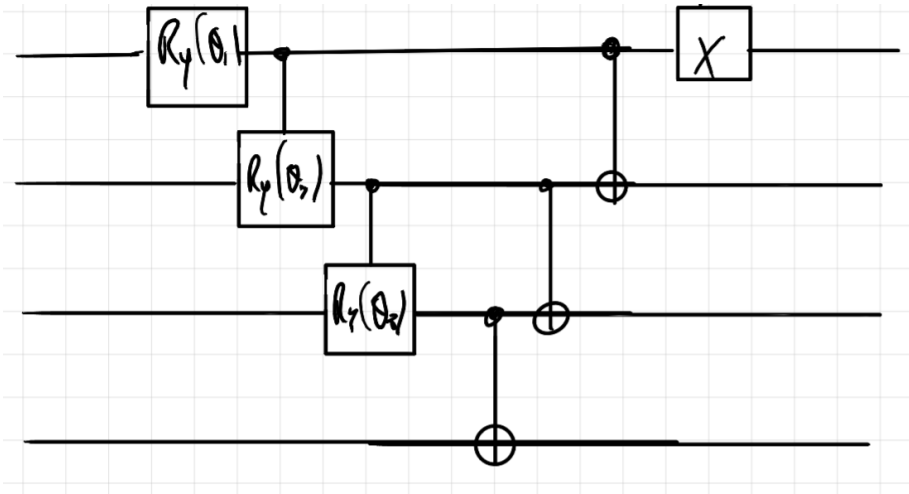
- Vertex-based polygons (α states).
- Edge-based polygons (β states).

$$|\alpha_i\rangle = \frac{1}{\sqrt{\sum_j w_{ij}}} \sum_j \sqrt{w_{ij}}\,|v_{ij}\rangle, \qquad |\beta_{ij}\rangle = \frac{|v_{ij}\rangle + |v_{ji}\rangle}{\sqrt{2}}$$



Backup: Implementing W_α

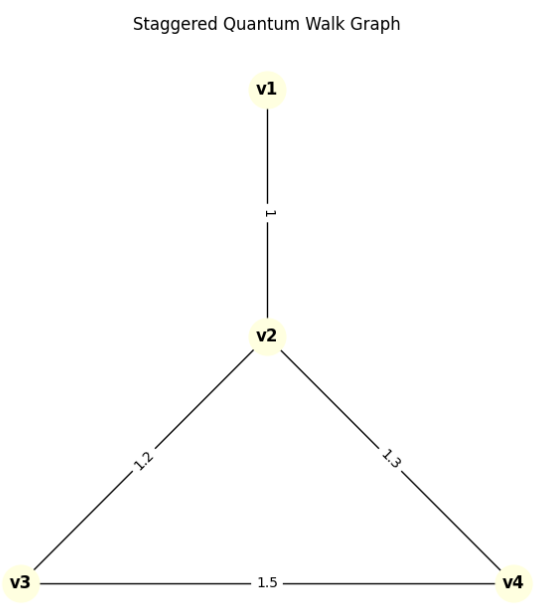




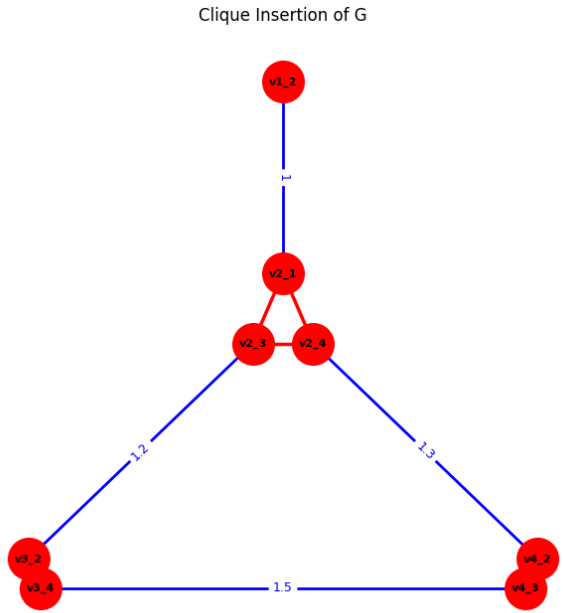
$$W_\alpha = \bigotimes_{\alpha_k \in \alpha} U_{\alpha_k} C^{s-1} Z U_{\alpha_k}^\dagger$$

where $U_{\alpha_k} = \tilde{U}_{\alpha_k} X^s$. For $s = 4$, for example, $\tilde{U}_{\alpha_k}$ is defined as in the figure above. U_{α_k} is defined such that $U_{\alpha_k}|1\cdots 1_s\rangle = |\alpha_k\rangle$.

Backup: Example Graph and Clique Insertion



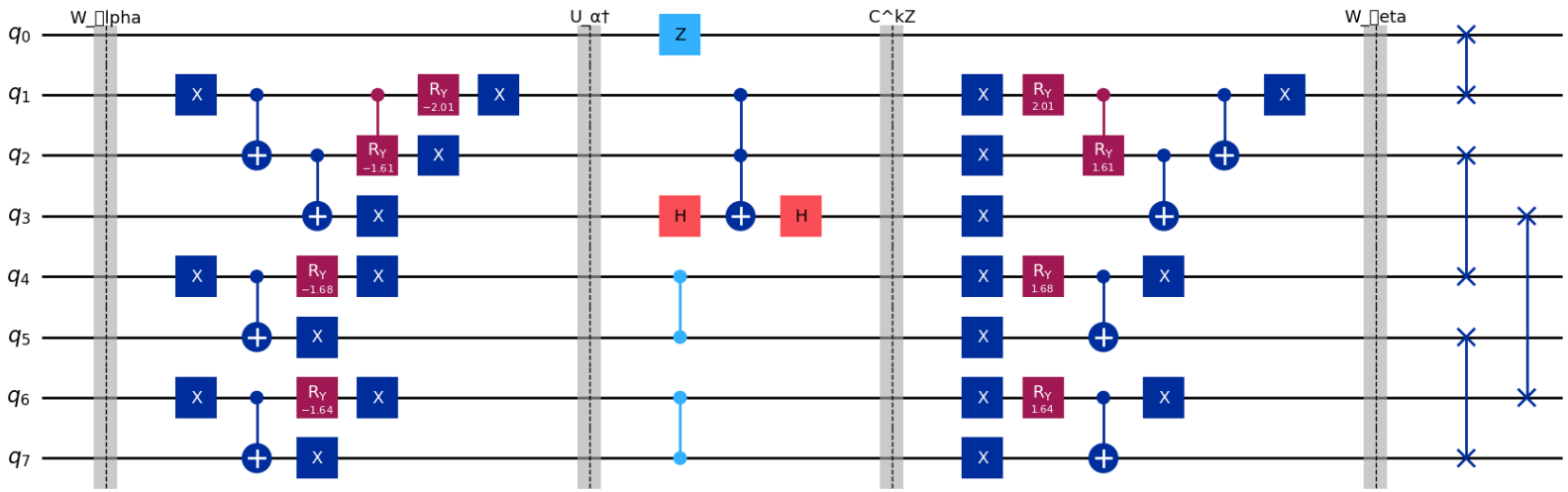
Weighted graph G



After the clique insertion operator $\mathcal{C}(G)$

Backup: Circuit Implementation of $W = W_\beta W_\alpha$

$$W_\alpha = I - 2\sum_k |\alpha_k\rangle\langle\alpha_k|, \qquad W_\beta = I - 2\sum_{ij} |\beta_{ij}\rangle\langle\beta_{ij}|$$



Backup: Community Detection via Quantum Walk

Step 1 — Limiting edge-probability vector

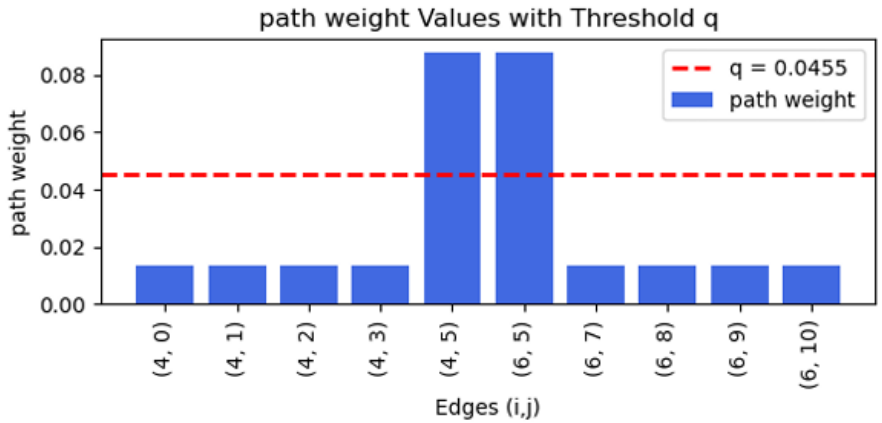
Run the Extended Staggered Quantum Walk and compute the time-averaged distribution:

$$\Pi = \lim_{T\to\infty} \frac{1}{T+1} \sum_{t=0}^{T} P_t$$

We iterate until:

$$\left\| \Pi^{(T)} - \Pi^{(T-1)} \right\| < \varepsilon$$

Each edge e gets probability $p(i \to j) = p(v_{i,j}) + p(v_{j,i})$.



Inter-community edges (bridges) accumulate higher probability mass

Backup: Path Weight & Community Assignment

Step 2 — Path weight between vertices

Given Π, define the path weight between u and v along the shortest path $u = u_0, e_1, \ldots, e_n, u_n = v$:

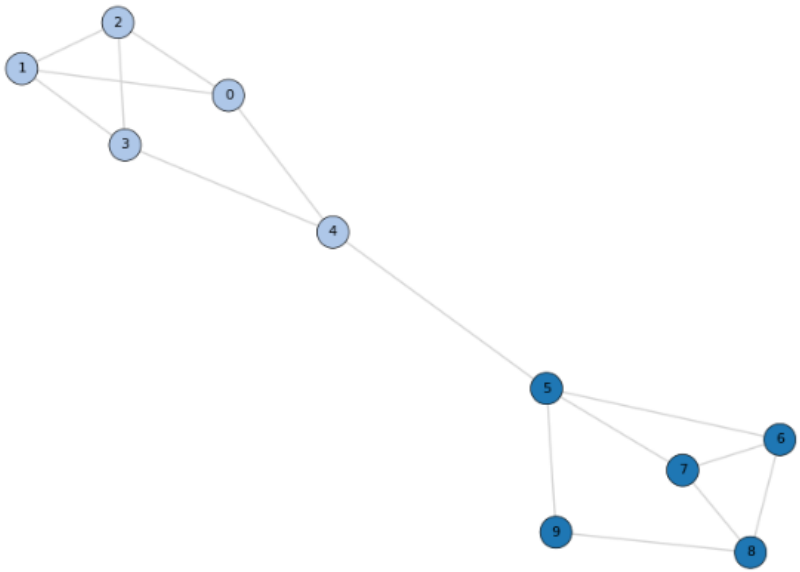
$$\mathrm{PW}(u, v) = \frac{1}{d_G(v)} \prod_{i=1}^{n} p(e_i)$$

The factor $1/d_G(v)$ gives preference to high-degree nodes.

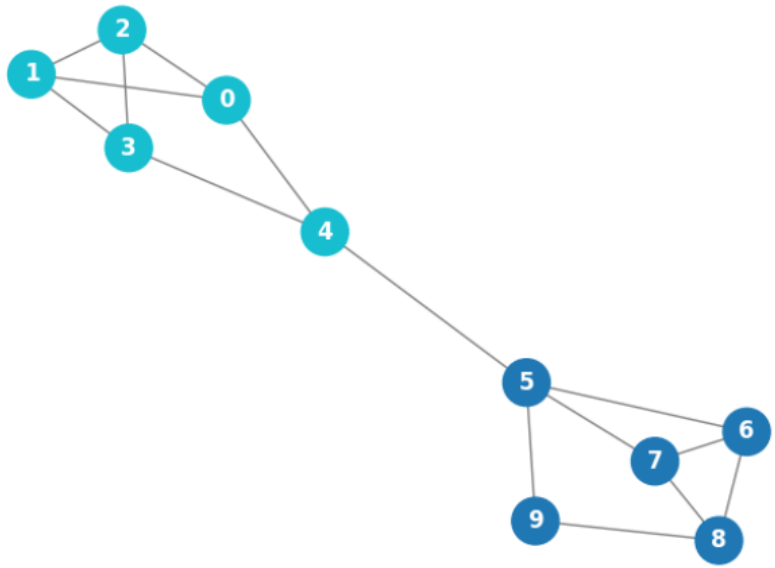
Step 3 — Community assignment

- Sort vertices by degree (descending).
- Seed a community from the highest-degree unassigned vertex v_i.
- Add neighbor v_j to $C(v_i)$ if $\mathrm{PW}(v_i, v_j) < q$ (q: single tunable parameter).
- Refinement: remove $v_j \in C(v_i)$ if $d_{C(v_i)}(v_j) \leq d_G(v_j)/2$.

Backup: Caveman Graph — Sanity Check



Quantum Walk



L-VQE

Both recover the correct partition.

- QW: 750 steps to convergence of Π
- L-VQE: 700 cost function evaluations

Caveman graphs are a trivial case just to do a quick test on both cases.